PAC-MAN

(DATA STRUCTURES PROJECT)

Presented To :

DR.Rasha saleh

Team Members:

NAME	IDENTIFICATION CODE	SECTION
MOHAMED SALEM ELSAYED SAAD ELDIN	91241041	3
MOSTAFA KHALED MOHAMED KAMEL MABROUK ELMARAKBY	91240757	3
MOSTAFA SAMY MOSTAFA SHEHATA MOHAMMED HATAB	91240759	3
YEHIA MOHAMED YEHIA MOHAMED	91240857	3
Mohamed Essam Badr Mohamed	91240674	3
mohamed magdy amin naem	91240683	3
mohamed mahmoud mohamed asaad	91240689	3

Table of Contents

Introduction

FatMan is a simple 2D arcade-style game inspired by classic maze games. The main idea of the game is to control a character moving inside a maze, collecting items while avoiding enemies. The game focuses on quick decision making, movement control, and basic strategy.
The player must explore the map, collect all available points, and survive as long as possible without getting caught. The design is kept simple to make the game easy to understand but still fun and challenging.

Game Overview

In FatMan, the player controls a character inside a maze filled with small collectible items (pellets). The goal is to collect all pellets to complete the level.
At the same time, there are enemies moving inside the maze trying to catch the player. The player must avoid these enemies while navigating through walls and paths.
Some special items give the player temporary advantages, such as the ability to defeat enemies for a short time.
The game ends when the player loses all lives or when all levels are completed.
The game also includes simple keyboard controls for better interaction and user experience.

Game Rules

- The player can move in four directions: up, down, left, and right.
- The character cannot pass through walls.
- The player collects points by eating pellets scattered in the maze.
- Each collected pellet increases the score.
- Special pellets (power-ups) give temporary abilities, like defeating enemies.
- Enemies move inside the maze and try to catch the player.
- If an enemy touches the player, the player loses a life.
- The game continues until all lives are lost.
- The level is completed when all pellets are collected.
- Some maps may include portals that teleport the player from one side to another.
- The player can press R to start or restart the game.
- The player can press Q to quit the game at any time.

Mohamed Essam Task

- Input Management and Movement Queuing and Maze design

Movement logic.cpp

```
#include "logic.h"     // Game logic header (ROWS, COLS, function declarations)
#include <iostream>
using namespace std;

// Move Pacman based on direction input
void movePacman(int& x, int& y, char dir, char maze[ROWS][COLS + 1], int& score) {
  int nx = x, ny = y;  // New temporary position

  if (dir == 'w') ny--; // Move up
  else if (dir == 's') ny++; // Move down
  else if (dir == 'a') nx--; // Move left
  else if (dir == 'd') nx++; // Move right

  // Portal teleport (wrap around horizontally on row 9)
  if (ny == 9) {
    if (nx >= COLS) nx = 0;      // Exit right -> enter left
    else if (nx < 0) nx = COLS - 1; // Exit left -> enter right
  }

  // Boundary check (outside maze)
  if (nx < 0 || nx >= COLS || ny < 0 || ny >= ROWS) return;

  // Wall check
  if (maze[ny][nx] == '#') return;

  // Blocked door or special wall
  if (maze[ny][nx] == '=') return;

  // Apply movement if valid
  x = nx;
  y = ny;
}

}
```

Mohamed Essam Task

- Input Management and Movement Queuing and Maze design

Movement logic.cpp

```
/ Check and handle pellet eating
bool eatPellet(int x, int y, char maze[ROWS][COLS + 1], int& score) {
 // Out of bounds check
 if (x < 0 || x >= COLS || y < 0 || y >= ROWS) return false;
 // Normal pellet
 if (maze[y][x] == '.') {
 maze[y][x] = ' '; // Remove pellet
 score += 10; // Add score
 return false;
 }
 // Power pellet
 if (maze[y][x] == 'o') {
 maze[y][x] = ' '; // Remove power pellet
 score += 100; // Big score bonus
 return true; // Power mode activated
 }
 return false; // Nothing eaten
}
// Reset maze to original layout
void resetMaze(char maze[ROWS][COLS + 1]) {
 // Original maze layout stored in array
 const char originalMaze[ROWS][COLS + 1] = {
 " ################### ",
 " #........#........# ",
 " #o##.###.#.###.##o# ",
 " #.................# ",
 " #.##.#.#####.#.##.# ",
 " #....#...#...#....# ",
 " ####.### # ###.#### ",
 " #.# 0 #.# ",
"#####.# ##=## #.#####",
 " . #123# . ",
"#####.# ##### #.#####",
 " #.# #.# ",
 " ####.# ##### #.#### ",
 " #........#........# ",
 " #.##.###.#.###.##.# ",
 " #o.#.....P.....#.o# ",
 " ##.#.#.#####.#.#.## ",
 " #....#...#...#....# ",
 " #.######.#.######.# ",
 " #.................# ",
```

Mohamed Essam Task

- Input Management and Movement Queuing and Maze design

Movement logic.cpp

```
" ################### "
};
// Copy original maze into current maze
for (int i = 0; i < ROWS; i++)
for (int j = 0; j <= COLS; j++)
maze[i][j] = originalMaze[i][j];
}
// Initialize maze and place ghost house
void initializeMazeWithGhostHouse(char maze[ROWS][COLS + 1]) {
resetMaze(maze); // Load default maze
maze[8][10] = '='; // Add ghost house gate
```

header file

```
#pragma once
#ifndef LOGIC_H
#define LOGIC_H

const int ROWS = 21;  // maze height
const int COLS = 21;  // maze width

void movePacman(int& x, int& y, char dir, char maze[ROWS][COLS + 1], int& score); // move Pac-Man
bool eatPellet(int x, int y, char maze[ROWS][COLS + 1], int& score);         // eat pellet + score
void resetMaze(char maze[ROWS][COLS + 1]);                          // reset maze layout
void initializeMazeWithGhostHouse(char maze[ROWS][COLS + 1]);              // setup maze + ghost house

#endif
```

Comments on Code

```
* PAC-MAN MOVEMENT & MAZE INTERACTION SYSTEM
* - Handles Pac-Man movement based on user input (W, A, S, D)
* - Prevents movement through walls and restricted tiles
* - Implements portal teleportation for wrap-around gameplay
* - Ensures player stays within maze boundaries
* - Updates Pac-Man's position only if the move is valid
* - Detects and processes normal pellet consumption
* - Updates the score when pellets are eaten
* - Identifies power pellets and triggers special game effects
* - Resets the maze to its original layout when needed
* - Initializes special elements like the ghost house door for proper setup
```

Mohamed Essam Task

- Input Management and Movement Queuing and Maze design

Parts in the main cpp

```
static char maze[ROWS][COLS + 1];  // game map

static int pacX = 9, pacY = 15;    // Pac-Man current position
static int prevPacX = 9, prevPacY = 15; // previous position for rendering update

static char currentDir = 'd';     // current movement direction (right)
static char queuedDir = 'd';      // buffered next direction from input
```

try to get packman in queued direction

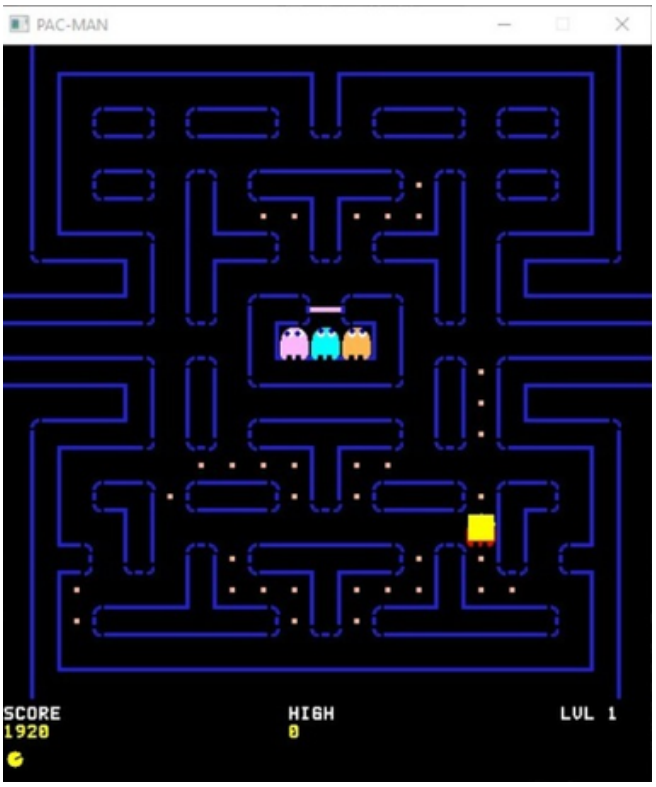
```
static bool canMove(char dir) {
  // Function checks if Pac-Man can move in the given direction
  // Returns true → movement is allowed
  // Returns false → movement is blocked

  int nx = pacX, ny = pacY;
  // Create temporary variables for the next position
  // Start from current Pac-Man position

  if (dir == 'w') ny--;
  // If direction is up → decrease row (move up)

  else if (dir == 's') ny++;
  // If direction is down → increase row (move down)

  else if (dir == 'a') nx--;
  // If direction is left → decrease column (move left)

  else if (dir == 'd') nx++;
  // If direction is right → increase column (move right)

  if (ny == 9) {
    // Check if Pac-Man is in tunnel row (row 9)

    if (nx >= COLS) nx = 0;
    // If moving beyond right edge → wrap to left side (portal)

    else if (nx < 0) nx = COLS - 1;
    // If moving beyond left edge → wrap to right side (portal)
  }

  if (nx < 0 || nx >= COLS || ny < 0 || ny >= ROWS) return false;
  // Boundary check:
  // If next position is outside maze → movement is not allowed

  return maze[ny][nx] != '#' && maze[ny][nx] != '=';
  // Final check:
  // Return true ONLY if next cell is NOT:
  // '#' → wall (blocked)
  // '=' → ghost door (blocked for Pac-Man)
}
```

Mohamed Essam Task

- Input Management and Movement Queuing and Maze design

packman movement

```
pacMoveAcc += dt;
// accumulate time

if (pacMoveAcc >= PAC_MOVE_INTERVAL) {
// check if it's time to move

  pacMoveAcc = 0;
  // reset timer

  prevPacX = pacX;
  prevPacY = pacY;
  // save previous position

  // try buffered input first
  if (canMove(queuedDir))
    currentDir = queuedDir;
  // update direction if valid

  movePacman(pacX, pacY, currentDir, maze, score);
  // move Pac-Man
}
```



Keyboard Input Event Handler

```
while (SDL_PollEvent(&e)) // check input events
{
  if (e.type == SDL_QUIT) // close window
    { running = false; keepPlaying = false; break; }

  if (e.type == SDL_KEYDOWN) // key pressed
  {
    SDL_Keycode k = e.key.keysym.sym;

    if (k == SDLK_w || k == SDLK_UP) queuedDir = 'w'; // up
    if (k == SDLK_s || k == SDLK_DOWN) queuedDir = 's'; // down
    if (k == SDLK_a || k == SDLK_LEFT) queuedDir = 'a'; // left
    if (k == SDLK_d || k == SDLK_RIGHT) queuedDir = 'd'; // right
  }
}
```

- GHOST CORE SYSTEM

GHOST SYSTEM HEADER

(IDENTITIES & BEHAVIOR &CONFIG)

```
#pragma once
#ifndef GHOST_H
#define GHOST_H

#include "logic.h"

enum GhostType {
  BLINKY,
  PINKY,
  INKY,
  CLYDE
};

enum GhostMode {
  SCATTER,
  CHASE,
  FRIGHTENED
};

struct Ghost {
  int x, y;
  int prevX, prevY;
  int scatterX, scatterY;
  GhostType type;
  GhostMode mode;
  char prevDir;
  bool active;
  int frightenedTimer;
  int releaseTimer;
};

struct DifficultyParams {
  int ghostMoveInterval;
  int frightenedDuration;
  int scatterTicks;
  int chaseTicks;
  int pinkyRelease;
  int inkyRelease;
  int clydeRelease;
};
```

• GHOST CORE SYSTEM

GHOST SYSTEM HEADER CONTINUE

(IDENTITIES & BEHAVIOR &CONFIG)

```
void initGhosts(Ghost ghosts[4], const DifficultyParams& params);

DifficultyParams getDifficultyParams(int level);

void moveGhost(Ghost& ghost, char maze[ROWS][COLS + 1],
 int pacX, int pacY, char pacDir,
 const Ghost* blinky);

void frightenGhosts(Ghost ghosts[4], int frightenedDuration);

void updateFrightenedTimers(Ghost ghosts[4]);

void updateModeCycle(Ghost ghosts[4], int tick, const DifficultyParams& params);

void updateReleaseTimers(Ghost ghosts[4]);

#endif
```

Comments on Code

* GHOST SYSTEM OVERVIEW (AI + BEHAVIOR CONTROL)
* - Defines all ghost identities: BLINKY, PINKY, INKY, CLYDE
* - Each ghost has a unique targeting AI strategy
* - Defines ghost behavior states:
* → SCATTER: move to assigned corners
* → CHASE: actively pursue Pac-Man
* → FRIGHTENED: escape mode after power pellet
* - Ghost struct stores full runtime state (position, mode, timers)
* - Includes difficulty system for scaling gameplay per level
* - Controls movement speed, frightened duration, and release timing
* - Function declarations manage:
* → initialization of ghosts
* → AI movement logic
* → frightened state handling
* → mode switching system (scatter/chase cycle)
* → ghost house release timing

- GHOST CORE SYSTEM

GHOST MATH &DIRECTION & MOVEMENT HELPERS

```
#include "ghost.h"
#include <cstdlib>

static int distSq(int x1, int y1, int x2, int y2) {
  return (x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1);
}

static char reverseDir(char dir) {
  if (dir == 'w') return 's';
  if (dir == 's') return 'w';
  if (dir == 'a') return 'd';
  if (dir == 'd') return 'a';
  return ' ';
}

static void applyDir(char dir, int x, int y, int& nx, int& ny) {
  nx = x; ny = y;
  if (dir == 'w') ny--;
  else if (dir == 's') ny++;
  else if (dir == 'a') nx--;
  else if (dir == 'd') nx++;
}

static void applyTeleport(int& x, int& y) {
  if (y == 9) {
    if (x >= COLS) x = 0;
    else if (x < 0) x = COLS - 1;
  }
}

static bool isInsideHouse(int x, int y) {
  return (y >= 8 && y <= 10 && x >= 8 && x <= 12);
}

static bool isWalkable(char maze[ROWS][COLS + 1], int x, int y, bool insideHouse) {
  if (y == 9 && (x < 0 || x >= COLS)) return true;
  if (x < 0 || x >= COLS || y < 0 || y >= ROWS) return false;
  char tile = maze[y][x];
  if (tile == '#') return false;
  if (tile == '=' && !insideHouse) return false;
  return true;
}
```

• GHOST CORE SYSTEM

Comments on Code

* GHOST CORE HELPERS SYSTEM
* - distSq: fast distance comparison without sqrt (performance optimization)
* - reverseDir: returns opposite direction (used for AI reversal logic)
* - applyDir: simulates movement without modifying state
* - applyTeleport: handles tunnel wrap-around on row 9
* - isInsideHouse: detects ghost house area
* - isWalkable: validates if tile can be moved into

GHOST INITIALIZATION & DIFFICULTY

```
void initGhosts(Ghost ghosts[4], const DifficultyParams& params) {
  ghosts[BLINKY] = { 10, 7, 10, 7, 18,  1, BLINKY, CHASE,    'd', true, 0, 0 };
  ghosts[PINKY]  = { 9,  9,  9, 9,  2,  1, PINKY,  SCATTER,  'w', true, 0, params.pinkyRelease };
  ghosts[INKY]   = { 10, 9, 10, 9, 18, 19, INKY,   SCATTER,  's', true, 0, params.inkyRelease };
  ghosts[CLYDE]  = { 11, 9, 11, 9,  2, 19, CLYDE,  SCATTER,  's', true, 0, params.clydeRelease };
}

DifficultyParams getDifficultyParams(int level) {
  DifficultyParams p;
  if (level == 1) {
    p = { 200, 40, 10, 200, 30, 60, 90 };
  }
  else if (level <= 4) {
    p = { 160, 30,  7, 300, 20, 40, 60 };
  }
  else if (level <= 8) {
    p = { 130, 15,  5, 400, 10, 20, 30 };
  }
  else if (level <= 12) {
    p = { 110,  5,  3, 500,  5, 10, 15 };
  }
  else {
    p = { 90,  0,  1, 999,  2,  4,  6 };
  }
  return p;
}
```

• GHOST CORE SYSTEM

Comments on Code

* GHOST INITIALIZATION & DIFFICULTY SYSTEM
* - Sets starting positions and states for all 4 ghosts
* - Defines each ghost personality (BLINKY, PINKY, INKY, CLYDE)
* - Assigns scatter corners and release timers
* - Difficulty system scales:
* → movement speed
* → frightened duration
* → scatter/chase timing
* → ghost house release speed
* - Higher levels = faster and more aggressive AI

CHASE TARGETING SYSTEM

```
static void getChaseTarget(const Ghost& ghost, int pacX, int pacY, char pacDir,
  const Ghost* blinky, int& tx, int& ty) {

  switch (ghost.type) {

  case BLINKY:
    tx = pacX; ty = pacY;
    break;

  case PINKY: {
    tx = pacX; ty = pacY;
    for (int i = 0; i < 4; i++) {
      int nx, ny;
      applyDir(pacDir, tx, ty, nx, ny);
      tx = nx; ty = ny;
    }
    break;
  }

  case INKY: {
    int pivX = pacX, pivY = pacY;
    for (int i = 0; i < 2; i++) {
      int nx, ny;
      applyDir(pacDir, pivX, pivY, nx, ny);
      pivX = nx; pivY = ny;
    }
```

• GHOST CORE SYSTEM

CHASE TARGETING SYSTEM CONTINUE

```
if (blinky) {
tx = pivX + (pivX - blinky->x);
ty = pivY + (pivY - blinky->y);
}
else {
tx = pacX; ty = pacY;
}
break;
}

case CLYDE:
if (distSq(ghost.x, ghost.y, pacX, pacY) > 64) {
tx = pacX; ty = pacY;
}
else {
tx = ghost.scatterX; ty = ghost.scatterY;
}
break;
}

if (tx < 0) tx = 0; if (tx >= COLS) tx = COLS - 1;
if (ty < 0) ty = 0; if (ty >= ROWS) ty = ROWS - 1;
}
```

Comments on Code

* GHOST CHASE AI SYSTEM
* - Implements unique AI behavior for each ghost:
* → BLINKY: direct chase
* → PINKY: predicts 4 tiles ahead
* → INKY: uses Blinky + Pac-Man vector (complex flanking)
* → CLYDE: switches between chase and scatter based on distance
* - Creates distinct personalities for each ghost
* - Ensures targets stay within maze bounds

- GHOST CORE SYSTEM

GHOST MOVEMENT ENGINE

```
void moveGhost(Ghost& ghost, char maze[ROWS][COLS + 1],
  int pacX, int pacY, char pacDir, const Ghost* blinky) {

  ghost.prevX = ghost.x;
  ghost.prevY = ghost.y;

  if (!ghost.active) return;

  bool inside = isInsideHouse(ghost.x, ghost.y);

  if (inside) {
    if (ghost.releaseTimer > 0) return;
    if (ghost.x < 10)     { ghost.x++; ghost.prevDir = 'd'; }
    else if (ghost.x > 10) { ghost.x--; ghost.prevDir = 'a'; }
    else                   { ghost.y--; ghost.prevDir = 'w'; }
    return;
  }

  int targetX, targetY;
  if (ghost.mode == SCATTER) {
    targetX = ghost.scatterX; targetY = ghost.scatterY;
  }
  else if (ghost.mode == CHASE) {
    getChaseTarget(ghost, pacX, pacY, pacDir, blinky, targetX, targetY);
  }
  else {
    targetX = -1; targetY = -1;
  }

  const char dirs[] = { 'w','s','a','d' };
  char forbidden = reverseDir(ghost.prevDir);
  char bestDir = ' ';
  int bestDist = -1;
  int validCount = 0;

  for (int i = 0; i < 4; i++) {
    char d = dirs[i];
    if (d == forbidden) continue;
```

- GHOST CORE SYSTEM

GHOST MOVEMENT ENGINE CONTINUE

```
int nx, ny;
 applyDir(d, ghost.x, ghost.y, nx, ny);
 applyTeleport(nx, ny);

 if (!isWalkable(maze, nx, ny, inside)) continue;

 if (ghost.mode == FRIGHTENED) {
 validCount++;
 if (rand() % validCount == 0) bestDir = d;
 }
 else {
 int d2 = distSq(nx, ny, targetX, targetY);
 if (bestDir == ' ' || d2 < bestDist) {
 bestDist = d2;
 bestDir = d;
 }
 }
 }

 if (bestDir == ' ') {
 char rev = reverseDir(ghost.prevDir);
 int nx, ny;
 applyDir(rev, ghost.x, ghost.y, nx, ny);
 applyTeleport(nx, ny);
 if (isWalkable(maze, nx, ny, inside)) bestDir = rev;
 }

 if (bestDir != ' ') {
 int nx, ny;
 applyDir(bestDir, ghost.x, ghost.y, nx, ny);
 applyTeleport(nx, ny);
 ghost.x = nx;
 ghost.y = ny;
 ghost.prevDir = bestDir;
 }
}
```

• GHOST CORE SYSTEM

Comments on Code

* GHOST MOVEMENT ENGINE
* - Handles full ghost movement logic each tick
* - Manages ghost house exit behavior
* - Switches between SCATTER / CHASE / FRIGHTENED modes
* - Evaluates all possible directions each step
* - Prevents reverse movement unless stuck
* - Uses AI target system for intelligent chasing
* - Includes fallback logic for dead ends

FRIGHTENED &MODE SYSTEM

```
void frightenGhosts(Ghost ghosts[4], int frightenedDuration) {
  if (frightenedDuration == 0) return;
  for (int i = 0; i < 4; i++) {
    if (!ghosts[i].active) continue;
    if (ghosts[i].mode == FRIGHTENED) continue;
    ghosts[i].mode = FRIGHTENED;
    ghosts[i].frightenedTimer = frightenedDuration;
    ghosts[i].prevDir = reverseDir(ghosts[i].prevDir);
  }
}

void updateFrightenedTimers(Ghost ghosts[4]) {
  for (int i = 0; i < 4; i++) {
    if (ghosts[i].mode != FRIGHTENED) continue;
    ghosts[i].frightenedTimer--;
    if (ghosts[i].frightenedTimer <= 0) {
      ghosts[i].mode = CHASE;
      ghosts[i].frightenedTimer = 0;
    }
  }
}

void updateReleaseTimers(Ghost ghosts[4]) {
  for (int i = 0; i < 4; i++)
    if (ghosts[i].releaseTimer > 0)
      ghosts[i].releaseTimer--;
}
```

• GHOST CORE SYSTEM

Comments on Code

* GHOST STATE CONTROL SYSTEM
* - frightenGhosts: activates blue vulnerable mode
* - updateFrightenedTimers: handles countdown & reset to CHASE
* - updateReleaseTimers: controls ghost house exit timing
* - Ensures proper state transitions during gameplay
* - Manages power pellet behavior and ghost vulnerability

SCATTER &CHASE CYCLE SYSTEM

```
void updateModeCycle(Ghost ghosts[4], int tick, const DifficultyParams& params) {
  int s = params.scatterTicks;
  int c = params.chaseTicks;
  int cycleLen = 2 * s + c;
  int phase = tick % cycleLen;

  GhostMode newMode;
  if (phase < s)       newMode = SCATTER;
  else if (phase < s + c)  newMode = CHASE;
  else             newMode = SCATTER;

  for (int i = 0; i < 4; i++) {
    if (ghosts[i].mode == FRIGHTENED) continue;
    if (ghosts[i].mode != newMode) {
      ghosts[i].mode = newMode;
      ghosts[i].prevDir = reverseDir(ghosts[i].prevDir);
    }
  }
}
```

Comments on Code

* GHOST MODE CYCLE SYSTEM
* - Alternates between SCATTER and CHASE phases
* - Based on global tick timing
* - Creates classic Pac-Man AI behavior pattern
* - Frightened mode overrides cycle logic
* - Forces direction reversal on mode switch

• GHOST SPAWN SYSTEM

GHOST SPAWN MANAGEMENT HEADER

```
#pragma once
#ifndef GHOST_SPAWN_H
#define GHOST_SPAWN_H

#include "ghost.h"

struct GhostSpawnPosition {
  int x, y;
  int releaseDelay;
};

struct SpawnManager {
  GhostSpawnPosition blinkySpawn;
  GhostSpawnPosition pinkySpawn;
  GhostSpawnPosition inkySpawn;
  GhostSpawnPosition clydeSpawn;
};

SpawnManager getSpawnPositions(int level);

void resetAllGhostsToSpawn(Ghost ghosts[4], const SpawnManager& spawnMgr, const DifficultyParams& params);

void resetGhostToSpawn(Ghost& ghost, const SpawnManager& spawnMgr, const DifficultyParams& params);

#endif
```

Comments on Code

* GHOST SPAWN MANAGEMENT SYSTEM
* - Defines spawn data structure for each ghost (position + release delay)
* - Groups all ghost spawn configurations using SpawnManager
* - Supports level-based spawn adjustments (faster releases at higher levels)
* - getSpawnPositions:
* → returns spawn positions and release timings per level
* - resetAllGhostsToSpawn:
* → resets all ghosts when starting a new life or level
* - resetGhostToSpawn:
* → resets a single ghost after being eaten
* - Ensures consistent and controlled ghost re-entry into gameplay

• GHOST SPAWN SYSTEM

SPAWN POSITION SETUP

```
#include "ghost_spawn.h"

SpawnManager getSpawnPositions(int level) {
  SpawnManager manager;

  manager.blinkySpawn = { 10, 7,  0 };
  manager.pinkySpawn  = { 9,  9, 30 };
  manager.inkySpawn   = { 10, 9, 60 };
  manager.clydeSpawn  = { 11, 9, 90 };

  if (level > 5) {
    manager.pinkySpawn.releaseDelay  = 20;
    manager.inkySpawn.releaseDelay   = 40;
    manager.clydeSpawn.releaseDelay  = 60;
  }
  return manager;
}
```

Comments on Code

* GHOST SPAWN POSITION SYSTEM
* - Defines initial spawn positions for all ghosts
* - Blinky starts outside the ghost house and moves immediately
* - Pinky, Inky, and Clyde start inside the house with delays
* - Release delays control when each ghost enters gameplay
* - Higher levels reduce delays, increasing difficulty
* - Ensures controlled and progressive ghost entry into the game

• GHOST SPAWN SYSTEM

FULL GHOST RESET

```
void resetAllGhostsToSpawn(Ghost ghosts[4], const SpawnManager& spawnMgr, const DifficultyParams& params) {

  ghosts[BLINKY].x = spawnMgr.blinkySpawn.x;
  ghosts[BLINKY].y = spawnMgr.blinkySpawn.y;
  ghosts[BLINKY].prevX = ghosts[BLINKY].x;
  ghosts[BLINKY].prevY = ghosts[BLINKY].y;
  ghosts[BLINKY].mode = CHASE;
  ghosts[BLINKY].active = true;
  ghosts[BLINKY].releaseTimer = spawnMgr.blinkySpawn.releaseDelay;
  ghosts[BLINKY].frightenedTimer = 0;

  ghosts[PINKY].x = spawnMgr.pinkySpawn.x;
  ghosts[PINKY].y = spawnMgr.pinkySpawn.y;
  ghosts[PINKY].prevX = ghosts[PINKY].x;
  ghosts[PINKY].prevY = ghosts[PINKY].y;
  ghosts[PINKY].mode = SCATTER;
  ghosts[PINKY].active = true;
  ghosts[PINKY].releaseTimer = params.pinkyRelease;
  ghosts[PINKY].frightenedTimer = 0;

  ghosts[INKY].x = spawnMgr.inkySpawn.x;
  ghosts[INKY].y = spawnMgr.inkySpawn.y;
  ghosts[INKY].prevX = ghosts[INKY].x;
  ghosts[INKY].prevY = ghosts[INKY].y;
  ghosts[INKY].mode = SCATTER;
  ghosts[INKY].active = true;
  ghosts[INKY].releaseTimer = params.inkyRelease;
  ghosts[INKY].frightenedTimer = 0;

  ghosts[CLYDE].x = spawnMgr.clydeSpawn.x;
  ghosts[CLYDE].y = spawnMgr.clydeSpawn.y;
  ghosts[CLYDE].prevX = ghosts[CLYDE].x;
  ghosts[CLYDE].prevY = ghosts[CLYDE].y;
  ghosts[CLYDE].mode = SCATTER;
  ghosts[CLYDE].active = true;
  ghosts[CLYDE].releaseTimer = params.clydeRelease;
  ghosts[CLYDE].frightenedTimer = 0;
}
```

• GHOST SPAWN SYSTEM

Comments on Code

* FULL GHOST RESET SYSTEM
* - Resets all ghosts to their initial spawn positions
* - Blinky starts outside the house in CHASE mode
* - Other ghosts start inside the house in SCATTER mode
* - Release timers are set based on difficulty parameters
* - Resets movement history and frightened state
* - Used when starting a new level or after losing a life

SINGLE GHOST RESET

```
void resetGhostToSpawn(Ghost& ghost, const SpawnManager& spawnMgr, const DifficultyParams& params) {
  switch (ghost.type) {
  case BLINKY:
    ghost.x = spawnMgr.blinkySpawn.x;
    ghost.y = spawnMgr.blinkySpawn.y;
    ghost.releaseTimer = spawnMgr.blinkySpawn.releaseDelay;
    break;
  case PINKY:
    ghost.x = spawnMgr.pinkySpawn.x;
    ghost.y = spawnMgr.pinkySpawn.y;
    ghost.releaseTimer = params.pinkyRelease;
    break;
  case INKY:
    ghost.x = spawnMgr.inkySpawn.x;
    ghost.y = spawnMgr.inkySpawn.y;
    ghost.releaseTimer = params.inkyRelease;
    break;
  case CLYDE:
    ghost.x = spawnMgr.clydeSpawn.x;
    ghost.y = spawnMgr.clydeSpawn.y;
    ghost.releaseTimer = params.clydeRelease;
    break;
  }

  ghost.prevX = ghost.x;
  ghost.prevY = ghost.y;
  ghost.mode = SCATTER;
  ghost.active = true;
  ghost.frightenedTimer = 0;
}
```

• GHOST SPAWN SYSTEM

Comments on previous Code

* SINGLE GHOST RESPAWN SYSTEM
* - Resets one ghost after being eaten by Pac-Man
* - Uses ghost type to determine correct spawn position
* - Applies appropriate release delay per ghost
* - Resets movement history and state
* - Forces ghost into SCATTER mode on respawn
* - Ensures smooth reintegration into gameplay

MAIN GAME LOOP – GHOST UPDATE & TIMING SYSTEM

```
ghostMoveAcc += dt;

if (ghostMoveAcc >= difficulty.ghostMoveInterval) {
  ghostMoveAcc = 0;

  updateModeCycle(ghosts, now / 100, difficulty);
  updateFrightenedTimers(ghosts);
  updateReleaseTimers(ghosts);

  for (int g = 0; g < 4; g++) {
    if (ghosts[g].releaseTimer <= 0)
      moveGhost(ghosts[g], maze, pacX, pacY, currentDir, &ghosts[BLINKY]);
  }
}
```

Comments on previous Code

* GHOST MOVEMENT TICK SYSTEM
* - Uses time accumulation to control ghost movement timing
* - Moves ghosts only when accumulated time reaches movement interval
* - Movement speed is controlled by difficulty parameters
* - Updates all ghost timers each tick:
* → Mode cycle (SCATTER / CHASE switching)
* → Frightened timers (blue state duration)
* → Release timers (ghost house exit timing)
* - Iterates through all ghosts and moves only active ones
* - Ensures smooth, time-based movement independent of frame rate

- COLLISION TASK

COLLISION RESPONSE HANDLING

```
static int checkGhostCollision() {
  for (int i = 0; i < 4; i++) {
    if (!ghosts[i].active) continue;
    if (ghosts[i].x == pacX && ghosts[i].y == pacY)       return i;
    if (ghosts[i].x == prevPacX && ghosts[i].y == prevPacY
      && ghosts[i].prevX == pacX && ghosts[i].prevY == pacY) return i;
  }
  return -1;
}
```

Comments on Code

* Checks for collisions between Pac-Man and the 4 ghosts.
* - Skips inactive ghosts (e.g., those still in the cage).
* - Checks for direct hits (exact same X, Y coordinates).
* - Checks for "pass-through" hits (Pac-Man and a ghost swap positions in the same frame).
* - Returns the index of the colliding ghost (0-3), or -1 if no collision occurred.

- COLLISION TASK

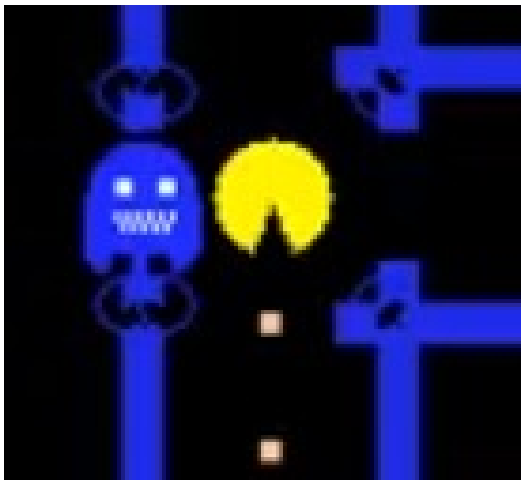
HANDLE GHOST COLLISION OUTCOME

```
static bool handleCollision(int idx, SDL_Renderer* sdlRen, Renderer& ren) {
  if (idx == -1) return false;

  if (ghosts[idx].mode == FRIGHTENED) {
    score += 200;

    int eX = ghosts[idx].x, eY = ghosts[idx].y;
    playEatenAnimation(sdlRen, ren, eX, eY);

    // Spawn position per ghost type
    int newX = 10, newY = 7;
    switch (ghosts[idx].type) {
    case BLINKY: newX = 10; newY = 7; break;
    case PINKY:  newX = 9; newY = 9; break;
    case INKY:   newX = 10; newY = 9; break;
    case CLYDE:  newX = 11; newY = 9; break;
    }

    animateGhostReturn(sdlRen, ren, ghosts[idx], eX, eY, newX, newY);

    ghosts[idx].mode = CHASE;
    ghosts[idx].frightenedTimer = 0;
    ghosts[idx].active = true;
    ghosts[idx].x = ghosts[idx].prevX = newX;
    ghosts[idx].y = ghosts[idx].prevY = newY;
    ghosts[idx].releaseTimer = 0;
    return false;
  }

  // Normal hit - lose a life
  lives--;
  playDeathAnimation(sdlRen, ren);
  return true;
}
```

• COLLISION TASK

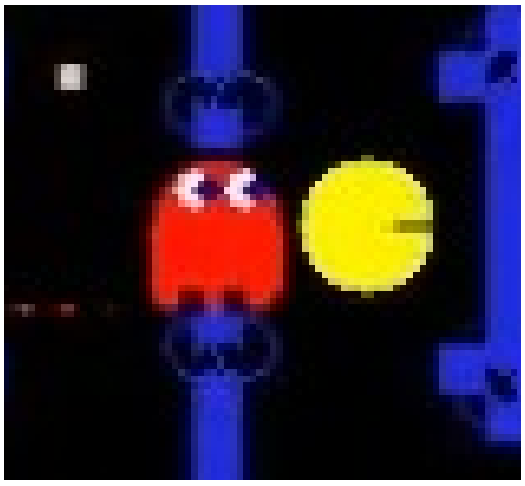
Comments on pervious Code

* Handles the outcome of a collision between Pac-Man and a ghost.
* * Scenario 1: The ghost is FRIGHTENED (Pac-Man is empowered).
* - Increases the player's score by 200.
* - Records the collision coordinates to play the "ghost eaten" animation.
* - Determines the specific target coordinates inside the cage based on the ghost's type.
* - Plays the animation of the ghost's eyes returning to the cage.
* - Resets the ghost's state (changes mode back to CHASE, resets timers so it exits at normal speed, and keeps it active).
* - Returns false (Pac-Man did not lose a life).
* Scenario 2: Normal hit (The ghost is not frightened).
* - Decrements Pac-Man's life count.
* - Plays Pac-Man's death animation.
* - Returns true (Pac-Man lost a life).



FRIGHTENED MODE

(Pac-Man is empowered)



CHASE MODE

(The ghost is not frightened)

- COLLISION TASK

COLLISION CHECK & GAME FLOW DECISION

```
int caught = checkGhostCollision();
        if (caught != -1) {
          if (handleCollision(caught, sdlRen, ren)) {
            // Lost a life
            if (lives > 0) {
              respawnPacman();
              showReadyScreen(sdlRen, ren);
              lastTick = SDL_GetTicks();
              pacMoveAcc = ghostMoveAcc = 0;
            }
          }
        }
```

Comments on Code

* Main game loop collision logic:
* - Calls checkGhostCollision() to see if Pac-Man hit a ghost (returns 0-3 for a ghost, or -1 for no hit).
* - If a collision occurred (caught != -1), it passes the ghost index to handleCollision() to resolve the interaction.
* - If handleCollision() returns true (meaning the ghost was in CHASE mode and Pac-Man died):
* - Checks if the player has remaining lives.
* - If so, respawns Pac-Man and shows the "Ready" screen.
* - Resets the game timers (lastTick) and movement accumulators to prevent sudden speed spikes when gameplay resumes.

Yehia Mohamed Task

GAME STATE FINALIZATION, DOT SYSTEM & DOCUMENTATION

READY SCREEN FUNCTION

```
static void showReadyScreen(SDL_Renderer* sdlRen, Renderer& ren) {
  Uint32 start = SDL_GetTicks();
  SDL_Event e;

  while (SDL_GetTicks() - start < 2000) {
    while (SDL_PollEvent(&e)) {
      if (e.type == SDL_QUIT)
        exit(0);
    }

    ren.drawFrame(maze, pacX, pacY, currentDir, ghosts,
      score, highScore, lives, level,
      SDL_GetTicks(), true, false, false);

    SDL_Delay(16);
  }
}
```

READY SCREEN

Comments on Code

* READY SCREEN SYSTEM
* - Displays the "READY" screen for 2 seconds before gameplay starts
* - Uses SDL_GetTicks() to track elapsed time
* - Runs a loop that lasts exactly 2000 milliseconds
* - Handles window close events to safely exit the game
* - Continuously renders frames with READY flag enabled
* - Uses SDL_Delay(16) to maintain ~60 FPS rendering
* - Provides a smooth transition before the game begins

Yehia Mohamed Task

GAME STATE FINALIZATION, DOT SYSTEM & DOCUMENTATION

DOTS REMAIN CHECK

```
static bool dotsRemain() {
  for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
      if (maze[r][c] == '.' || maze[r][c] == 'o')
        return true;
    }
  }
  return false;
}
```

Comments on Code

* DOTS REMAIN CHECK SYSTEM
* - Scans the entire maze grid for remaining dots
* - '.' represents normal dots
* - 'o' represents power pellets
* - Returns true if any collectible item is still present
* - Returns false when all dots are cleared
* - Used to detect level completion
* - Ensures the game progresses only after clearing the maze

Yehia Mohamed Task

GAME STATE FINALIZATION, DOT SYSTEM & DOCUMENTATION

Report Documentation

I was responsible for preparing the full project report, where I worked closely with each team member to thoroughly understand their individual modules and implementation details. This included reviewing their code, discussing design decisions, and clarifying logic to ensure complete and accurate documentation.

Based on this, I created a well-structured organizational report that covers the entire system from A to Z — starting from the overall architecture and design flow, down to detailed explanations of core functionalities such as game logic, ghost AI, rendering, timing system, and user interaction.

The report ensures that all components are clearly connected, consistently documented, and easy to understand, reflecting the full workflow of the project in a professional and organized manner.

Mostafa Samy Task

• GAME ENGINE CORE (STATE MANAGEMENT & LEVEL FLOW)

GAME STATE VARIABLES & DRAW FUNCTION

```
static int score = 0;
static int highScore = 0;
static int lives = 3;
static int level = 1;

void drawGame(Renderer& ren, Uint32 currentTime) {
  ren.drawFrame(maze, pacX, pacY, currentDir, ghosts,
    score, highScore, lives, level,
    currentTime,
    false,
    false,
    false
  );
}
```

Comments on Code

* CORE GAME STATE & RENDERING SYSTEM
* - Stores main game variables (score, high score, lives, level)
* - drawGame() is a wrapper around renderer
* - Responsible for drawing a single frame of the game
* - Passes all game state + timing to render engine

Mostafa Samy Task

• GAME ENGINE CORE (STATE MANAGEMENT & LEVEL FLOW)

LEVEL COMPLETE SYSTEM

```
static void handleLevelComplete(SDL_Renderer* sdlRen, Renderer& ren) {

  for (int f = 0; f < 6; f++) {

    SDL_SetRenderDrawBlendMode(sdlRen, SDL_BLENDMODE_BLEND);

    Uint8 alpha = (f % 2 == 0) ? 120 : 0;
    SDL_SetRenderDrawColor(sdlRen, 255, 255, 255, alpha);

    SDL_Rect overlay = { 0, 0, WIN_W, ROWS * TILE };
    SDL_RenderFillRect(sdlRen, &overlay);

    SDL_SetRenderDrawBlendMode(sdlRen, SDL_BLENDMODE_NONE);
    SDL_RenderPresent(sdlRen);

    SDL_Delay(200);
  }

  level++;

  initializeMazeWithGhostHouse(maze);
  difficulty = getDifficultyParams(level);
  spawnManager = getSpawnPositions(level);
  initGhosts(ghosts, difficulty);
  respawnPacman();

  showReadyScreen(sdlRen, ren);
}
```

Comments on Code

* LEVEL COMPLETION & GAME PROGRESSION SYSTEM
* - Triggered when all dots are eaten
* - Plays flash screen animation effect
* - Increases level counter
* - Reloads maze and difficulty settings
* - Resets ghosts and player position
* - Shows READY screen before next level starts

Mostafa Samy Task

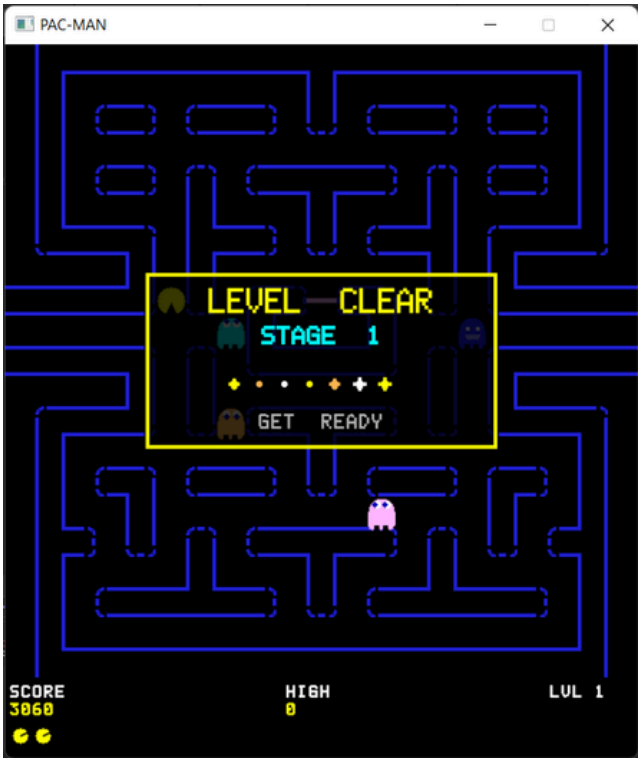
• GAME ENGINE CORE (STATE MANAGEMENT & LEVEL FLOW)

LIFE MANAGEMENT

```
void loseLife(SDL_Renderer* sdlRen, Renderer& ren) {

  lives--;

  playDeathAnimation(sdlRen, ren);
}
```

Comments on Code

* PLAYER LIFE SYSTEM
* - Decreases lives when collision happens
* - Triggers death animation for feedback
* - Can be extended for sound/UI updates



LEVEL COMPLETED

Mostafa Elmarakby Task

• GAME STATE MANAGEMENT & GAME OVER HANDLING

Respawn Pac-Man

```
static void respawnPacman()
{
 pacX = 9; pacY = 15;
 currentDir = 'd'; queuedDir = 'd';
 resetAllGhostsToSpawn(ghosts, spawnManager, difficulty);
}
```

Comments on Code

* Respawns Pac-Man to the starting position after losing a life or starting a new round.
 * - Resets Pac-Man's coordinates to the default spawn point (9, 15).
 * - Sets both current direction and queued direction to 'd' (default direction – usually right).
 * - Resets all ghosts back to their spawn positions using the current difficulty settings.

Restart Game

```
static void restartGame()
{
 score = 0; lives = 3; level = 1;
 initializeMazeWithGhostHouse(maze);
 difficulty = getDifficultyParams(level);
 spawnManager = getSpawnPositions(level);
 initGhosts(ghosts, difficulty);
 respawnPacman();
}
```

Comments on Code

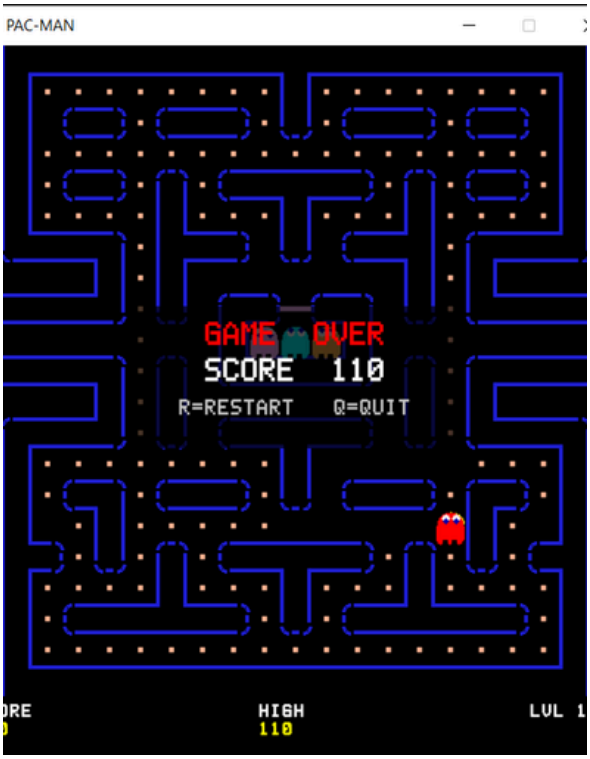
* Restarts the entire game from the beginning.
 * - Resets score, lives, and level to their initial values (0, 3, 1).
 * - Reinitializes the maze including the ghost house.
 * - Updates difficulty parameters based on the starting level.
 * - Retrieves spawn positions for the current level.
 * - Initializes all ghosts using the current difficulty settings.
 * - Calls respawnPacman() to place Pac-Man and ghosts at their starting positions.

Mostafa Elmarakby Task

- GAME STATE MANAGEMENT & GAME OVER HANDLING

Show Game Over Screen

```
static bool showGameOverScreen(SDL_Renderer* sdlRen, Renderer& ren)
 {
 SDL_Event e;
 Uint32 start = SDL_GetTicks();
 while (true)
 {
 while (SDL_PollEvent(&e))
{
 if (e.type == SDL_QUIT) return false;
 if (e.type == SDL_KEYDOWN)
{
 SDL_Keycode k = e.key.keysym.sym;
 if (k == SDLK_r) return true;
 if (k == SDLK_q || k == SDLK_ESCAPE) return false;
 }
 }
 ren.drawFrame(maze, pacX, pacY, currentDir, ghosts,
 score, highScore, lives, level,
 SDL_GetTicks(), false, true, false);
 SDL_Delay(16);
 }
}
```



Comments on Code

* GAME OVER screen
* - Displays the final game state with a "GAME OVER" overlay.
* - Waits for player input in a continuous loop.
* - Press 'R' → restarts the game (returns true).
* - Press 'Q' or 'ESC' → quits the game (returns false).
* - Closing the window also exits the game.

GAME GRAPHICS & ANIMATION SYSTEM

(DEATH ANIMATION - GHOST RETURN - FRAME RENDERING)

Delta Time Calculation

```
Uint32 now = SDL_GetTicks();
int dt = (int)(now - lastTick);
lastTick = now;
if (dt > 100) dt = 100;
```

Comments on Code

* Calculates the time difference (dt) between the current frame and the previous frame.
* - SDL_GetTicks() returns the current time in milliseconds since the program started.
* - Subtracting lastTick from the current time gives the elapsed time (dt) since the last frame.
* - dt is used to update movement accumulators, ensuring time-based movement.
* - When an accumulator reaches its threshold (e.g., 140ms), a movement step is executed and the accumulator resets.
* - The clamp (if dt > 100 → dt = 100) prevents sudden large jumps if the game freezes or is paused.
* - This approach ensures consistent game speed regardless of frame rate or hardware performance.

Play Death Animation

```
static void playDeathAnimation(SDL_Renderer* sdlRen, Renderer& ren) {
 for (int i = 0; i < 7; i++) {
 Color col = (i % 2 == 0) ? Color{ 255,255,0,255 } : Color{ 0,0,0,255 };
 ren.drawFrame(maze, pacX, pacY, currentDir, ghosts,
 score, highScore, lives, level,
 SDL_GetTicks(), false, false, false);
 SDL_SetRenderDrawColor(sdlRen, col.r, col.g, col.b, 255);
 SDL_Rect r = { pacX * TILE + 2, pacY * TILE + 2, TILE - 4, TILE - 4 };
 SDL_RenderFillRect(sdlRen, &r);
 SDL_RenderPresent(sdlRen);
 SDL_Delay(150);
  }
 }
```

Mohamed Salem Task

GAME GRAPHICS & ANIMATION SYSTEM

(DEATH ANIMATION - GHOST RETURN - FRAME RENDERING)

Comments on pervious Code

* DEATH ANIMATION & LIFE LOSS BEHAVIOR
* - Triggered when Pac-Man is caught by a normal ghost
* - Immediately deducts one life from the player
* - Plays a flashing death animation for visual feedback
* - The full game frame is redrawn 7 times in a loop
* - Pac-Man's tile is overpainted alternately (yellow / black)
* based on even/odd loop iteration to create flashing effect
* - Each flash lasts 150ms
* - Total animation duration is approximately 1 second
* - After animation:
* → If lives remain: Pac-Man is respawned
* → If no lives remain: Game Over screen is triggered

Animate Ghost Return

```
static void animateGhostReturn(SDL_Renderer* sdlRen, Renderer& ren,
 Ghost& ghost,
 int startX, int startY,
 int endX, int endY) {

 const int STEPS = 12;

 for (int i = 0; i <= STEPS; i++) {
 ghost.x = startX + (endX - startX) * i / STEPS;
 ghost.y = startY + (endY - startY) * i / STEPS;

 ren.drawFrame(maze, pacX, pacY, currentDir, ghosts,
 score, highScore, lives, level,
 SDL_GetTicks(), false, false, false);

 SDL_RenderPresent(sdlRen);
 SDL_Delay(18);
 }

 ghost.x = endX;
 ghost.y = endY;
}
```

Mohamed Salem Task

GAME GRAPHICS & ANIMATION SYSTEM

(DEATH ANIMATION - GHOST RETURN - FRAME RENDERING)

Comments on pervious Code

* When Pac-Man eats a power pellet, all ghosts turn blue and become vulnerable for a limited time.
* - If Pac-Man collides with a blue ghost during this window, the ghost is eaten.
* - animateGhostReturn() is triggered to handle the ghost's return animation.
* - Instead of instantly respawning the ghost at its spawn point, it moves smoothly across the maze.
* - The movement is done using linear interpolation over 12 steps.
* - Each step updates the ghost position, redraws the full game frame, and waits 18ms.
* - Total animation duration is approximately 216ms.
* - The full game scene (maze, Pac-Man, remaining ghosts, HUD) stays visible and updated throughout.
* - This provides clear visual feedback that the ghost is traveling back to its house before re-entering gameplay.

Mohamed Salem Task

GAME GRAPHICS & ANIMATION SYSTEM

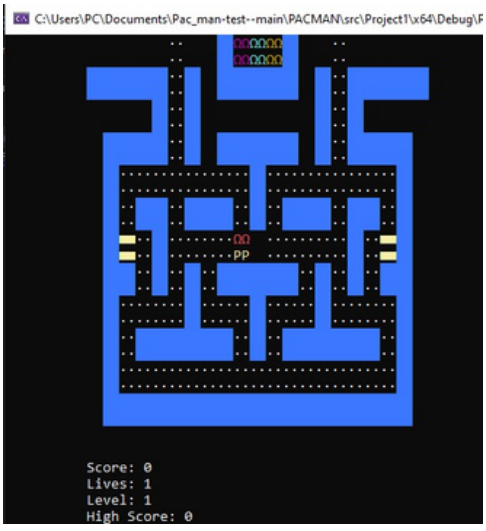
(DEATH ANIMATION - GHOST RETURN - FRAME RENDERING)

GAME GRAPHICS

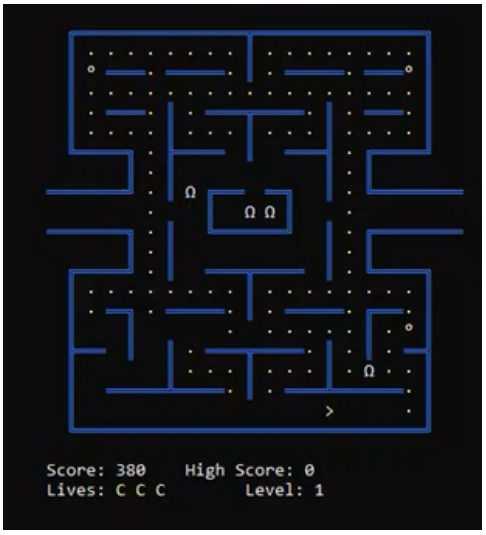
In this project, I was responsible for developing the complete graphics system from A to Z. This included designing and implementing the real-time rendering system that draws every frame of the game, including the maze, Pac-Man, ghosts, and all visual elements.

I also implemented all in-game visual effects and animations such as death animation, ghost return animation, flashing effects, and level transition visuals. The system ensures smooth frame updates and consistent visual output regardless of game speed or hardware performance.

Overall, the graphics system focuses on providing clear visual feedback to the player and maintaining a smooth and responsive visual experience throughout the game.



the foundational graphics version



the product using typical Cout/console procedures:

the product using SDL installations